



UNIVERSIDADE
FEDERAL DO CEARÁ

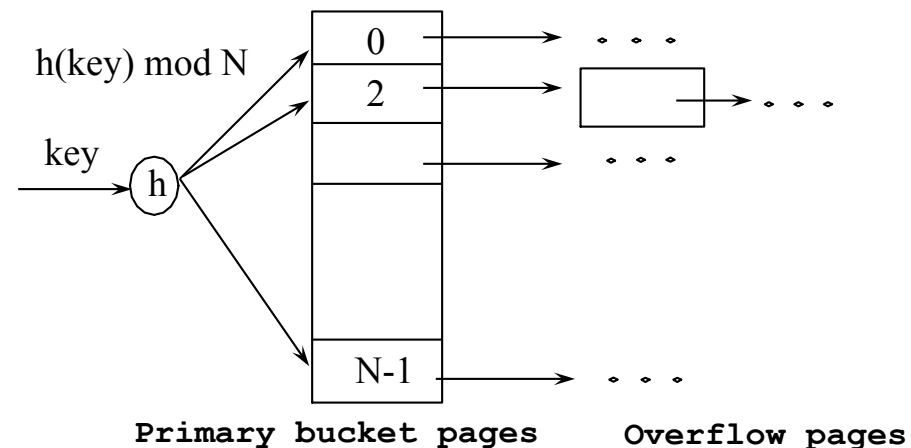


Índices Baseados em Hashing

Javam Machado
DC - UFC

Hash Estático

- Número de páginas primárias é fixo
- Páginas primárias são alocadas seqüencialmente, nunca são desalocadas
- Se necessário, páginas de overflow
- $h(k) \bmod N = \textit{bucket}$ ao qual a entrada de dados, com chave k , pertence. ($N = \text{num. de buckets}$)



Hash Estático

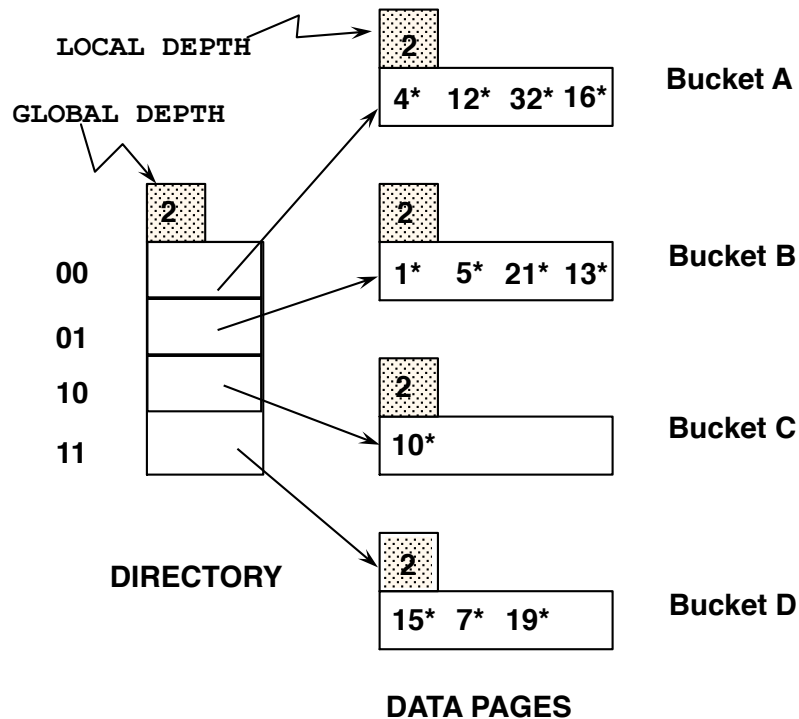
- Buckets contêm *entradas de dados*.
- *Função hash* atua na *chave de busca* do registro. Distribui valores no intervalo 0 ... N-1.
 - Na busca: $h(chave)$ retorna o Id do *bucket*.
 - busca no *bucket* pode ser sequencial
- Cadeias de páginas overflow podem tornar-se longas e comprometer a performance do índice.
 - *Hashing Extensível*: Técnica dinâmica para solucionar o problema.

Hash Extensível

- Situação: *Bucket* (página primária) cheio.
- Solução: Duplicar o número de *buckets* na área de páginas primária.
 - Ler e escrever todos os *buckets* é caro!
 - Idéia: Usar um diretório de ponteiros para buckets, duplicar o número de *buckets* através da duplicação do diretório, divide-se apenas o *bucket* cheio!
 - Diretório muito menor que um arquivo, sua duplicação é muito mais barata. Apenas um *bucket* é dividido. *Sem buckets overflow!*

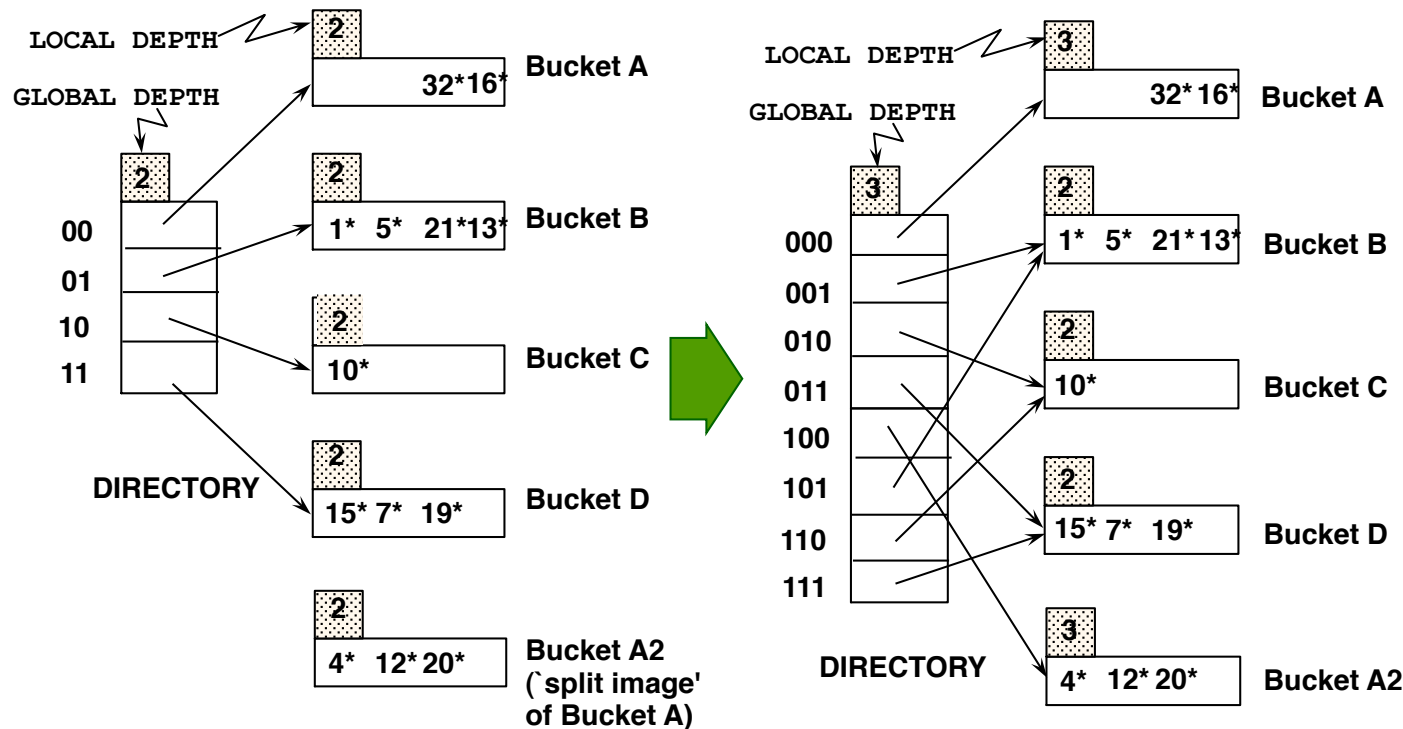
Hash Extensível

- Diretório de ponteiros é um vetor de tamanho $k = 4$.
- Para encontrar o *bucket* de um registro r , pegue os n bits menos significativos de $h(r)$, onde n é o *global depth*.
- Busca Registro de Chave de Busca = 5
- $h(r) = 101$ (em binário), r está no *bucket* apontado por 01.



Hash Extensível

- Inserção: Se bucket está cheio, divida-o (alocar novo bucket, redistribuir).
- Se necessário, duplicar o diretório.
- Exemplo: Inserção de $h(r) = 20$.



Hash Extensível

- Observações:

- 20 = binário 10100. Últimos **2** bits (00) dizem que r está em A ou em A2. Últimos **3** bits dizem especificamente em qual dos dois r está localizado.
- *Global depth* (profundidade global) do diretório: número max. de bits para identificar a qual *bucket* uma entrada pertence.
- *Local depth* (profundidade local) de um *bucket*: número de bits para determinar se uma entrada pertence a este *bucket*.

- Quando uma divisão de *buckets* causa a duplicação do diretório?

- Antes da inserção, *local depth* do *bucket* = *global depth*. Inserção faz com que *local depth* > *global depth*; diretório é duplicado e ponteiros são criados para fazer referência aos novos *buckets*.

Hash Extensível

- Se o diretório cabe na memória, buscas por igualdade são respondidas com apenas 1 acesso a disco; caso contrário, com 2.
- Arquivo: 100MB file, 100 bytes/registro, páginas de 4K contém 1.000.000 registros e 25.000 itens do diretório; diretório cabe em memória = probabilidade alta.
- Se a distribuição dos *valores de hash* não é homogênea, diretório pode crescer muito.
- Múltiplas entradas com o mesmo valor de *hash* causam problemas!
- **Remoção**: Se remoção de uma entrada torna *bucket* vazio, este pode ser fundido com o *bucket* seguinte. Se cada elemento do diretório aponta para o mesmo *bucket*, pode-se dividi-lo.